

Logic Programming & Declarative Paradigms

Liting Zhang

University of West Florida

Overview

- Declarative vs imperative thinking
- Predicate logic overview (facts, rules, quantifiers)
- Horn clauses and why they matter
- Prolog basics: facts, rules, queries
- Unification (the matching engine)
- Resolution and SLD: how Prolog “runs”
- Backtracking search: success, failure, choice points
- Expressing relations and constraints in Prolog
- Comparing Prolog to SQL, Datalog, and constraint DSLs
- Industry angle: rule engines, queries, constraint solvers

Learning goals

After this lecture, you should be able to

- Explain what “declarative” means and how it differs from imperative control flow
- Read basic predicate logic statements and map them to Horn clauses
- Write simple Prolog knowledge bases (facts + rules) and query them
- Explain unification and predict whether two terms unify
- Describe resolution and why backtracking search appears in Prolog
- Use recursion and lists to express relations (member, append, path)
- Control search (order of clauses/goals, cut, negation-as-failure)
- Use constraint logic programming (conceptual) for scheduling/puzzles
- Compare Prolog-style querying to SQL and rule/constraint DSLs

Core idea: describe relations, let the engine search

Logic programming mental model:

- You write relations (facts + rules)
- A query asks: “Is there a way to make this true?”
- The runtime finds values for variables (via unification) and explores possibilities (via backtracking)

Why it is different:

- You specify what is true, not how to compute it step-by-step
- Control flow emerges from search + rule application
- Debugging often means reasoning about search order

Where logic/declarative ideas show up

Even if you never ship Prolog, the ideas are everywhere:

- SQL: declare a set of rows you want
- Regular expressions: declare patterns, engine searches for matches
- Build systems: declare dependencies, tool figures out build order
- Policy engines: declare rules, engine decides allow/deny
- Constraint solvers: declare constraints, solver finds satisfying assignments

Declarative paradigms

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up

Imperative vs declarative

Imperative (typical C/Java/Python loops):

- Describe steps: update variables, iterate, branch
- Order matters: evaluation order is the program

Declarative (logic/SQL/constraints):

- Describe a property of the solution
- A solver/search engine chooses an execution strategy

Key question shift:

- Imperative: “How do I compute this?”
- Declarative: “What must be true of the answer?”

Declarative does not mean “no control”

Reality check:

- Declarative code often has an underlying engine (optimizer, solver, search)
- Performance can still depend on structure: indexes, rule order, constraints
- In Prolog, clause order and goal order affect search

Practical skill:

- Keep the program meaning declarative
- Learn engine behavior to avoid accidental slow search

Logic programming in one slide

A Prolog program is a set of Horn clauses:

- Facts: `parent(alice, bob).`
- Rules: `grandparent(X,Z) :- parent(X,Y), parent(Y,Z).`

You ask queries:

- `?- grandparent(alice, Z).`

The engine:

- Unifies terms to make goals match rule heads
- Uses backtracking to explore alternatives

Practice 0: declarative vs imperative

For each task, identify whether it is more naturally imperative, declarative, or both:

- Find all students with $\text{GPA} > 3.5$
- Compute $\text{Fibonacci}(40)$ quickly
- Assign exam times so no student has two exams at once
- Parse a log file and extract error lines

Practice 0 Answer: declarative vs imperative

- Find students with $\text{GPA} > 3.5$: declarative (SQL/filtering)
- Fibonacci(40) quickly: imperative (DP), or declarative with memoization
- Exam times without conflicts: declarative constraints (CSP/CLP)
- Parse log and extract error lines: both (regex/filters are declarative; scanning is imperative)

Predicate logic overview

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up

Predicates, terms, and variables

Building blocks:

- Constant: `alice`, `uwf`, `3`
- Variable: `X`, `Y` (capitalized in Prolog)
- Function symbol (term constructor): `pair(X,Y)`, `node(Left,Right)`
- Predicate: `parent(X,Y)`, `edge(U,V)`

Read:

- `parent(alice, bob)` is a proposition: either true or false in a model
- `parent(alice, X)` is a formula with a free variable (a query pattern)

Quantifiers (conceptual)

Predicate logic often uses:

- $\forall x$ (for all)
- $\exists x$ (there exists)

Example idea:

- “Everyone has a parent”: $\forall x \exists y \text{ parent}(y, x)$

Prolog viewpoint (informal):

- Variables in rules behave like “for all” in the head and body
- Queries ask “does there exist values that satisfy this?”

Implication and reading rules

Implication:

- $A \leftarrow B$ reads “A is true if B is true”
- In Prolog: $A :- B.$

Example:

- $grandparent(X, Z) \leftarrow parent(X, Y) \wedge parent(Y, Z)$
- Read: X is grandparent of Z if X is parent of Y and Y is parent of Z

Note:

- Rules define sufficient conditions, not necessarily necessary conditions

From English to logic (tiny examples)

English:

- If someone is a parent, they are an ancestor.
- If someone is an ancestor of Y , and Y is an ancestor of Z , then they are an ancestor of Z .

Logic-style:

- $ancestor(X, Y) \leftarrow parent(X, Y)$
- $ancestor(X, Z) \leftarrow ancestor(X, Y) \wedge ancestor(Y, Z)$

This is a recursive definition (a transitive closure).

Practice 1: translate to logic-style rules

Translate into rule form:

- “If X is a sibling of Y, then Y is a sibling of X.” (symmetry)
- “If X is a parent of Y, and Y is a parent of Z, then X is a grandparent of Z.”
- “If X is a professor and teaches course C, then X is an instructor of C.”

Use predicate names you like (e.g., sibling, parent, teaches).

Practice 1 Answer: translate to logic-style rules

One possible set:

- $sibling(X, Y) \leftarrow sibling(Y, X)$ is not a good definition (it loops)
- Better: treat `sibling` as given by facts, then add a derived predicate:
 - ▶ $sibling2(X, Y) \leftarrow sibling(X, Y)$
 - ▶ $sibling2(X, Y) \leftarrow sibling(Y, X)$
- $grandparent(X, Z) \leftarrow parent(X, Y) \wedge parent(Y, Z)$
- $instructor(X, C) \leftarrow professor(X) \wedge teaches(X, C)$

Key lesson: logic rules can accidentally create non-terminating recursion if used naively.

Horn clauses and Prolog

- 1 Declarative foundations
- 2 Prolog basics and unification**
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up

Horn clauses (why Prolog works)

A Horn clause (informal):

- A clause with at most one “positive” head atom
- Shape: $Head \leftarrow Body_1 \wedge \cdots \wedge Body_n$

Prolog programs:

- Facts: $Head \leftarrow true$
- Rules: $Head \leftarrow Body$

Why this restriction matters:

- Enables efficient proof search strategies (SLD resolution)
- Gives Prolog a clear operational semantics (goal reduction)

Prolog syntax cheat sheet

- Fact ends with a period .
- Rule uses :- and ends with .
- Variables start with capital letters or underscore
- Atoms start with lowercase (or are quoted)
- Query starts with ?-

```
1 parent(alice, bob).
2 parent(bob, carol).
3
4 grandparent(X, Z) :-
5     parent(X, Y),
6     parent(Y, Z).
7
8 ?- grandparent(alice, Who).
```

Example knowledge base: family

```
1  % facts
2  parent(alice, bob).
3  parent(bob, carol).
4  parent(alice, dana).
5  parent(dana, eric).
6
7  female(alice).
8  female(dana).
9  male(bob).
10 male(eric).
11 female(carol).
12
13 % rules
14 grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
15 mother(X, Y)      :- parent(X, Y), female(X).
16 father(X, Y)      :- parent(X, Y), male(X).
```

Reading:

- `mother(X,Y)` is true if X is a parent of Y and X is female

Queries return substitutions

Query patterns:

- `?- mother(X, bob).` “who is Bob’s mother?”
- `?- mother(alice, Y).` “who are Alice’s children?”

Conceptually, Prolog answers by finding a substitution:

- `X = alice`
- `Y = bob` or `Y = dana` (depending on query)

Multiple answers arise via backtracking.

Relational thinking: functions vs relations

In most languages, you write functions:

- input $\rightarrow$ output

In Prolog, you often write relations:

- multiple arguments can be “unknown”
- the same rule can be used in different directions

Example:

- `parent(alice, bob)` can answer both:
 - ▶ “Who are Alice’s children?”
 - ▶ “Who are Bob’s parents?”

Practice 2: write facts and basic queries

Write a tiny course database in Prolog:

- `teaches(prof, course).`
- `takes(student, course).`

Tasks:

- Add at least 3 teaches facts and 5 takes facts
- Query: which courses does a student take?
- Query: which students are in a given course?

Hint: start with atoms like `alice`, `cop4020`.

Practice 2 Answer: one possible solution

```
1 teaches(zhang, cop4020).
2 teaches(zhang, cop3530).
3 teaches(smith, cda3100).
4
5 takes(alice, cop4020).
6 takes(alice, cda3100).
7 takes(bob, cop4020).
8 takes(carol, cop3530).
9 takes(dana, cop4020).
10
11 % queries:
12 % ?- takes(alice, C).
13 % ?- takes(S, cop4020).
```

Unification

- 1 Declarative foundations
- 2 Prolog basics and unification**
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up

Unification: the matching engine

Unification answers:

- Can two terms be made equal by substituting variables?
- If yes, what is the most general substitution (MGU)?

Examples:

- `parent(alice, X)` unifies with `parent(alice, bob)` with $X=bob$
- `f(X, a)` unifies with `f(b, Y)` with $X=b$, $Y=a$

If unification fails, that search branch fails.

Term structure matters

Unification compares structure:

- Constants unify only if identical: `alice` with `alice`
- Compound terms unify if same functor + arity:
 - ▶ `edge(a,b)` can unify with `edge(X,Y)`
 - ▶ `edge(a,b)` cannot unify with `edge(a,b,c)`
- Variables unify with anything (subject to constraints)

Occurs check (conceptual)

The “occurs check” prevents infinite terms:

- $X = f(X)$ should not be allowed in strict unification
- Some Prolog systems omit full occurs-check for performance

What to remember for this course:

- Unification is usually “structural matching with variables”
- Be aware that some pathological cases exist

Unification examples with lists

Prolog list syntax:

- `[]` empty list
- `[a,b,c]` shorthand
- `[H|T]` head/tail form

```
1  % Unify [H/T] with [1,2,3]
2  % H = 1, T = [2,3]
3
4  % Unify [A,B|Rest] with [x,y,z]
5  % A = x, B = y, Rest = [z]
```

Practice 3: does it unify? (and with what?)

For each pair, decide unify or fail, and give bindings if unify:

- | | | | | |
|---|--|------------------------|------|---------------------|
| 1 | | 1) $f(X, a)$ | with | $f(b, Y)$ |
| 2 | | 2) $\text{edge}(X, X)$ | with | $\text{edge}(a, b)$ |
| 3 | | 3) $[H T]$ | with | $[1,2,3]$ |
| 4 | | 4) $[X, Y]$ | with | $[Y, X]$ |
| 5 | | 5) $g(X)$ | with | $f(X)$ |

Practice 3 Answer: does it unify?

- 1) unifies: $X=b$, $Y=a$
- 2) fails: would require $X=a$ and $X=b$ simultaneously
- 3) unifies: $H=1$, $T=[2,3]$
- 4) unifies with constraint $X=Y$; one MGU is $X=Y$ (they become same variable)
- 5) fails: different functors g vs f

Note for (4): $[X,Y] = [Y,X]$ implies $X=Y$.

Resolution and execution model

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search**
- 4 Programming with Prolog
- 5 Industry and wrap-up

Resolution (high-level)

Resolution is a proof rule for logic.

In Prolog terms (operational intuition):

- You have a goal list (a conjunction): $G1, G2, \dots$
- Pick the leftmost goal $G1$
- Find a clause whose head unifies with $G1$
- Replace $G1$ with that clause's body (new subgoals)

This is called SLD resolution (Selective Linear Definite clause resolution).

Prolog execution mantra

Prolog is often summarized as:

- unification + search

More operational detail:

- Depth-first search
- Left-to-right goals in a rule body
- Top-to-bottom clause order in the program

Consequence:

- Program meaning is logical, but performance/termination depends on order

A tiny proof trace (conceptual)

Program:

```
1 | parent(alice, bob).  
2 | parent(bob, carol).  
3 | grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

Query:

```
| ?- grandparent(alice, Z).
```

Goal reduction idea:

- Match `grandparent(alice,Z)` with rule head `grandparent(X,Z)`
- Bind `X=alice`; new goals: `parent(alice,Y)`, `parent(Y,Z)`
- Solve `parent(alice,Y)`: gives `Y=bob`
- Then solve `parent(bob,Z)`: gives `Z=carol`

Choice points

If multiple clauses can satisfy a goal:

- Prolog tries them in order
- Remembers alternatives as choice points
- If later goals fail, it backtracks to the most recent choice point

This is why one query can produce multiple answers.

Practice 4: predict a trace outcome

Consider:

```
1 | p(a) .  
2 | p(b) .  
3 | q(b) .  
4 | r(X) :- p(X), q(X) .
```

Questions:

- What does `?- r(X) .` return?
- In what order are candidates tried?

Practice 4 Answer: predict a trace outcome

Evaluate $r(X) :- p(X), q(X).$

- Try $p(X)$:
 - ▶ First fact $p(a)$ gives $X=a$
 - ▶ Then $q(a)$ fails (only $q(b)$ exists)
- Backtrack to $p(X)$:
 - ▶ Next fact $p(b)$ gives $X=b$
 - ▶ $q(b)$ succeeds

Result: one answer $X=b$.

Backtracking search

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search**
- 4 Programming with Prolog
- 5 Industry and wrap-up

Depth-first search: the good and the risky

Good:

- Simple mental model
- Produces answers incrementally
- Often efficient for “find one solution” tasks

Risky:

- Can get stuck exploring an infinite branch
- May be exponential without pruning/constraints

Practical message:

- Clause order and recursion structure matter for termination

Search tree intuition

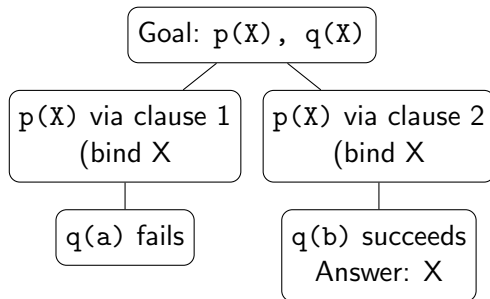
Think of evaluation as exploring a tree:

- Each goal can match multiple clauses $\rightarrow$ branching
- Each branch accumulates substitutions
- Failure prunes a branch

Depth-first:

- Explore one branch to completion, then backtrack

Backtracking diagram (conceptual)



Negation as failure

Prolog commonly uses `\+` Goal:

- Succeeds if Goal cannot be proven
- Fails if Goal can be proven

Example:

```
1 | bird(tweety).  
2 | penguin(pingu).  
3 |  
4 | bird(X) :- penguin(X).  
5 |  
6 | can_fly(X) :- bird(X), \+ penguin(X).
```

Caution:

- This is not classical logical negation; it depends on the closed-world assumption

The closed-world assumption

Many Prolog programs assume:

- Anything not provable from the knowledge base is considered false

This is often practical for databases and rule systems:

- If you didn't list it, we assume it doesn't exist

But it differs from open-world logic (e.g., knowledge graphs):

- “unknown” is not the same as “false”

Practice 5: backtracking with multiple answers

Consider:

```
1 | color(red).  
2 | color(green).  
3 | color(blue).  
4 |  
5 | pair(X, Y) :- color(X), color(Y), X \= Y.
```

Questions:

- How many answers does `?- pair(X, Y).` generate?
- What is the first answer (given top-to-bottom, left-to-right)?

Practice 5 Answer: backtracking with multiple answers

There are 3 colors, and $X \neq Y$ removes equal pairs:

- Total ordered pairs: $3 \times 3 = 9$
- Remove 3 equal pairs: $9 - 3 = 6$

First answer order:

- $X=\text{red}$ then $Y=\text{red}$ rejected, next $Y=\text{green}$
- First success: $X=\text{red}$, $Y=\text{green}$

Debugging and reasoning about search

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search**
- 4 Programming with Prolog
- 5 Industry and wrap-up

Why Prolog can feel “magical”

Beginners often experience:

- “I wrote the correct rules, but it loops”
- “I got an answer, but it took forever”
- “Changing order changes behavior”

The missing concept:

- Prolog is logical and operational (search strategy matters)

Two lenses:

- Declarative meaning: what is true
- Procedural meaning: how the engine searches

Common performance/termination tips

- Put base cases before recursive cases
- Avoid recursion patterns that never consume input
- Add visited checks on graphs
- Use constraints to prune early (when available)
- Be careful with `\+` when variables are not grounded

Groundness and negation

This can be dangerous:

```
1 | % "X is not a bird"  
2 | not_bird(X) :- \+ bird(X).
```

If you ask:

```
| ?- not_bird(X).
```

You are asking for all things that are not birds, which is not well-defined in a finite KB.

Safer pattern:

- Use negation after variables are bound (ground) by earlier goals

Practice 11: goal ordering for safety

Suppose:

```
1 | bird(tweety).  
2 | bird(woody).  
3 | penguin(pingu).  
4 | bird(X) :- penguin(X).
```

Which is safer?

- A: `can_fly(X) :- \+ penguin(X), bird(X).`
- B: `can_fly(X) :- bird(X), \+ penguin(X).`

Explain in terms of variable binding and search.

Practice 11 Answer: goal ordering for safety

Safer is B:

- `bird(X)` binds `X` to known birds/penguins first
- Then `\+ penguin(X)` checks whether that bound `X` can be proven to be a penguin

A is risky:

- `\+ penguin(X)` runs with `X` unbound; it asks “can I prove `penguin(X)` for some `X`?”
- Since `penguin(pingu)` exists, the negation fails immediately, making `can_fly(X)` fail for all `X`

Prolog programming patterns

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog**
- 5 Industry and wrap-up

Lists: the universal data structure

In classic Prolog, lists are central:

- Recursion over lists is a core pattern
- Many problems become “generate candidates” + “filter constraints”

Two common tasks:

- Define relations over lists: membership, append, length
- Use lists as paths, schedules, parse results, etc.

member/2 and append/3 (classic)

```
1 member(X, [X|_]).  
2 member(X, [_|T]) :- member(X, T).  
3  
4 append([], Ys, Ys).  
5 append([H|T], Ys, [H|Zs]) :- append(T, Ys, Zs).
```

Notes:

- member is relational: can ask “what is a member?” or “is this a member?”
- append can be used to split lists too

Relational power: splitting with append/3

Query:

```
| ?- append(Front, Back, [a,b,c]).
```

Conceptual answers (via backtracking):

- Front=[], Back=[a,b,c]
- Front=[a], Back=[b,c]
- Front=[a,b], Back=[c]
- Front=[a,b,c], Back=[]

This is hard to express as a single function in imperative style without extra loops.

Recursion patterns: structural recursion

For lists:

- Base case: empty list `[]`
- Recursive case: `[H|T]`

For trees:

- Base case: leaf
- Recursive case: node with subtrees

This aligns with inductive data definitions (like FP pattern matching).

Graph reachability: path/2

Facts (direct edges):

```
1 | edge(a, b).  
2 | edge(b, c).  
3 | edge(c, d).  
4 | edge(a, d).
```

Rules (reachability):

```
1 | path(X, Y) :- edge(X, Y).  
2 | path(X, Y) :- edge(X, Z), path(Z, Y).
```

This expresses transitive closure, but may loop on cyclic graphs.

Cycles and termination

If you add:

- `edge(d, b). (cycle)`

Then naive `path/2` can recurse forever.

Typical fix:

- carry a visited list and refuse to revisit nodes

Graph reachability with visited set

```
1 path(X, Y) :- path(X, Y, [X]).
2
3 path(X, Y, _) :-
4     edge(X, Y).
5
6 path(X, Y, Visited) :-
7     edge(X, Z),
8     \+ member(Z, Visited),
9     path(Z, Y, [Z|Visited]).
```

Now the recursion stops revisiting nodes.

Practice 6: list relations

Implement the following in Prolog:

- `last(X, List)`: X is the last element of List
- `prefix(P, L)`: P is a prefix of L
- `suffix(S, L)`: S is a suffix of L

Hint:

- Use recursion and/or `append/3`

Practice 6 Answer: list relations

One possible solution set:

```
1 | last(X, [X]).  
2 | last(X, [_|T]) :- last(X, T).  
3 |  
4 | prefix(P, L) :- append(P, _, L).  
5 | suffix(S, L) :- append(_, S, L).
```

Note:

- prefix/2 and suffix/2 become “generate all prefixes/suffixes” via backtracking.

Control: ordering goals and clauses

Two practical heuristics:

- Put more restrictive goals earlier (fail fast)
- Avoid generating huge candidate spaces before filtering

Example pattern:

- Bad: generate all lists, then test property
- Better: generate only the needed structure, apply constraints early

The cut (!) (control feature)

Cut commits to decisions made so far:

- It prunes choice points created earlier in the current predicate call

Example: deterministic max:

```
1 | max(X, Y, X) :- X >= Y, !.  
2 | max(_, Y, Y).
```

Intuition:

- If $X \geq Y$ succeeds, cut prevents trying the second clause later

Caution:

- Cut can make programs less purely declarative; use it to control search deliberately

Practice 7: where does cut change answers?

Consider:

```
1  p(1).  
2  p(2).  
3  
4  q(2).  
5  
6  r(X) :- p(X), !, q(X).  
7  r(X) :- p(X).
```

Question:

- What are the answers to $?- r(X)$.
- Explain the effect of the cut

Practice 7 Answer: where does cut change answers?

Evaluate first clause: $r(X) :- p(X), !, q(X).$

- $p(X)$ first gives $X=1$
- Cut commits to this $p(X)$ choice (no backtracking to $p(2)$)
- Then $q(1)$ fails
- Because of the cut, Prolog will not retry $p(X)$ with $X=2$ in that clause

Then Prolog tries the second clause: $r(X) :- p(X).$

- This yields $X=1$ then $X=2$

Final answers: $X=1$; $X=2$. The cut changed the search space of the first clause, not the second clause.

Using Prolog to express constraints

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog**
- 5 Industry and wrap-up

From pure relations to constraints

“Pure” Prolog:

- Uses unification + backtracking search
- Often does generate-and-test

Constraint Logic Programming (CLP) idea:

- Add constraint domains (integers, finite domains, reals)
- Propagate constraints to prune search early

Typical payoff:

- Scheduling, timetabling, resource allocation become natural

CLP(FD) flavor (conceptual example)

Many Prolog systems support finite-domain constraints (often via a library).

Example goal (read conceptually):

```
1  % X and Y are integers in 1..5
2  % X #\= Y means X not equal Y (constraint)
3  % X + Y #= 6 means X+Y equals 6
4
5  solve(X, Y) :-
6      X in 1..5,
7      Y in 1..5,
8      X #\= Y,
9      X + Y #= 6.
```

The solver narrows possibilities before brute-force enumeration.

Constraint mindset

Instead of:

- “Try all schedules and check conflicts”

You write:

- Variables represent unknown decisions
- Constraints express what must hold
- Labeling/search assigns values consistent with constraints

This is a classic declarative paradigm.

Practice 8: express constraints (no code needed)

You are scheduling 3 exams: A, B, C into time slots 1, 2, 3.
Constraints:

- A and B cannot be at the same time
- B and C cannot be at the same time
- A must be before C

Task:

- Write these as constraints over variables TA, TB, TC
- Find one valid schedule

Practice 8 Answer: express constraints

Constraints:

- $TA \neq TB$
- $TB \neq TC$
- $TA < TC$
- Each in $\{1,2,3\}$

One valid schedule:

- $TA=1, TB=2, TC=3$

Note:

- Constraints prune many impossible assignments automatically in CLP.

Using Prolog to express relations

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog**
- 5 Industry and wrap-up

Prolog feels like querying a database

Facts resemble rows; predicates resemble tables:

- `parent(alice,bob).` is like a row in table `Parent`

A query resembles a join:

- `grandparent(X,Z) :- parent(X,Y), parent(Y,Z).`

Conceptually:

- Prolog search behaves like a nested-loop join (unless optimized)
- SQL optimizers can reorder joins and use indexes automatically

Prolog “join” example

```
1 enrolled(alice, cop4020).
2 enrolled(alice, cda3100).
3 enrolled(bob, cop4020).
4
5 teaches(zhang, cop4020).
6 teaches(smith, cda3100).
7
8 student_of(Student, Prof) :-
9     enrolled(Student, Course),
10    teaches(Prof, Course).
```

Query:

```
?- student_of(alice, P).
```

SQL comparison (same idea)

SQL “student_of” query:

```
1 | SELECT e.student, t.prof
2 | FROM enrolled e
3 | JOIN teaches t ON e.course = t.course
4 | WHERE e.student = 'alice';
```

Compare:

- Prolog rule looks like a logical join specification
- SQL optimizer decides join strategy; Prolog uses a fixed search order by default

Practice 9: translate between Prolog and SQL (conceptually)

Given relation facts:

- `orders(Customer, Item).`
- `price(Item, Dollars).`

Tasks:

- Write a Prolog rule `spent(Customer, Dollars)` meaning Customer ordered an item with that price
- Write the analogous SQL join (no need to be perfect SQL syntax)

Practice 9 Answer: translate between Prolog and SQL

Prolog:

```
1 | spent(C, D) :-  
2 |     orders(C, Item),  
3 |     price(Item, D).
```

SQL idea:

```
1 | SELECT o.customer, p.dollars  
2 | FROM orders o  
3 | JOIN price p ON o.item = p.item;
```

Horn clauses beyond Prolog

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog**
- 5 Industry and wrap-up

Datalog (conceptual comparison)

Datalog is like a restricted Prolog:

- Often forbids complex function symbols (keeps it database-friendly)
- Typically uses set semantics (no duplicates) and guaranteed termination on finite DBs
- Common for program analysis, policy, graph queries

Prolog is more general:

- Can build arbitrary terms, can do deeper recursion, can be Turing-complete
- Termination is not guaranteed

Why query languages like restrictions

Restrictions enable:

- Optimizations (indexes, join reordering)
- Guaranteed termination (on finite input)
- Clear complexity bounds

Tradeoff:

- Less expressive for some computations

Logic as a foundation for configuration/policy

Many systems use a rule-based core:

- Policies: allow/deny decisions
- Compliance checks: must satisfy constraints
- Routing rules: choose target by conditions

Pattern:

- Facts about the world (requests, attributes)
- Rules derive conclusions (allowed, violations, actions)

Practice 10: write a policy as Horn-style rules

Write rules for:

- “A request is allowed if the user is an admin.”
- “A request is allowed if the user owns the resource.”
- “Otherwise denied.”

Use predicates like:

- `admin(User)`
- `owns(User, Resource)`
- `allow(User, Resource)`

Discussion: how does “otherwise denied” relate to the closed-world assumption?

Practice 10 Answer: policy as rules

Rules:

- `allow(U,R) :- admin(U).`
- `allow(U,R) :- owns(U,R).`

“Otherwise denied”:

- In Prolog-style closed world, if `allow(U,R)` cannot be proven, treat it as denied.
- This is a design choice; some systems want an explicit `deny` predicate.

Mini case study

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog**
- 5 Industry and wrap-up

Case study: course prerequisites

Model prerequisites as relations:

- `prereq(Course, PrereqCourse)`
- `can_take(Student, Course)`

Questions we want to ask:

- Which prerequisites are required (directly or indirectly)?
- What courses can a student take next?

This is graph reachability + set filtering.

Prerequisites example (conceptual Prolog)

```
1  prereq(cop3530, cop2510).
2  prereq(cop4020, cop3530).
3  prereq(cda3100, cop2510).
4
5  passed(alice, cop2510).
6  passed(alice, cda3100).
7
8  needs(X, Y) :- prereq(X, Y).
9  needs(X, Y) :- prereq(X, Z), needs(Z, Y).
10
11 eligible(S, C) :-
12     + passed(S, C),
13     + (needs(C, P), + passed(S, P)).
```

Read eligible: student has not passed the course, and there is no prerequisite P that the student has not passed.

Discussion: what is declarative here?

- `needs/2` is a clean declarative transitive closure definition
- `eligible/2` mixes declarative intent with operational negation patterns

Reality in industry:

- Many real rule systems blend declarative rules with evaluation strategy constraints

Practice 12: extend the case study

Add a rule:

- `recommended(S, C)` is true if `eligible(S, C)` and the course is a prerequisite (directly or indirectly) for `cop4020`

Discussion:

- What query would list recommended courses for Alice?

Practice 12 Answer: extend the case study

One possible idea:

- `recommended(S,C) :- eligible(S,C), needs(cop4020, C).`

Query:

- `?- recommended(alice, C).`

Interpretation:

- “Eligible and helps progress toward COP4020”

Industry angle

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up**

Rule engines

Rule engines in industry often look like:

- Facts: events, attributes, measurements
- Rules: if conditions then conclusions/actions

Use cases:

- pricing/discount policies
- fraud detection heuristics
- access control decisions
- alerting logic in observability

Why logic ideas help:

- business rules change frequently
- you want rules readable by domain experts

Query languages as declarative programming

SQL-like systems:

- relational databases
- distributed query engines
- graph queries (paths, reachability)

Typical differences vs Prolog:

- SQL has an optimizer and set semantics
- Prolog has a fixed evaluation order and backtracking

Both share the declarative idea: describe the result set.

Constraint solvers

Constraint solvers appear in:

- scheduling (airlines, exams, staff)
- routing and resource allocation
- verification (SAT/SMT solving)
- configuration solving (packages, dependencies)

Declarative approach:

- represent unknowns as variables
- add constraints
- solver finds assignments

When Prolog-style thinking is a good fit

Good fit:

- exploring combinations with constraints
- symbolic reasoning and relations (graphs, rules)
- quickly prototyping inference logic

Less ideal fit:

- heavy numeric computing
- problems needing strict performance predictability without careful tuning

Common compromise:

- write a small declarative core; integrate with imperative host code

Practice 13: pick the right paradigm

For each scenario, choose: Prolog-style rules, SQL-style queries, constraint solver, or imperative code.

- Filter transactions and compute daily totals
- Decide if a request is allowed based on dozens of policy conditions
- Assign 200 flights to gates with resource constraints
- Simulate a physics engine at 120 FPS

Practice 13 Answer: pick the right paradigm

One reasonable mapping:

- Filter + totals: SQL-style queries (or data pipeline)
- Policy decisions: Prolog-style rules / rule engine
- Flight-to-gate assignment: constraint solver / CLP
- Physics engine: imperative numeric code

Key point:

- Declarative tools are strongest when the problem is naturally a set of constraints/relations.

Review

- 1 Declarative foundations
- 2 Prolog basics and unification
- 3 Execution model and search
- 4 Programming with Prolog
- 5 Industry and wrap-up**

Summary: what to remember

- Declarative programming describes what, not step-by-step how
- Prolog programs are Horn clauses: facts + rules
- Execution is unification + backtracking search (depth-first, left-to-right)
- Unification finds substitutions; failure triggers backtracking
- Search order matters: clause order and goal order can affect termination/performance
- Lists + recursion are core Prolog patterns (member, append, path)
- Negation-as-failure assumes a closed world; use it carefully with variables
- Constraints (CLP) prune search and shine in scheduling/allocation problems
- SQL, rule engines, and constraint solvers are all declarative relatives

Review questions

- What does it mean for Prolog to be “relational” rather than “functional”?
- What is unification, and how does it differ from simple pattern matching?
- Why can clause order change termination or performance?
- When is negation-as-failure a reasonable modeling choice?
- Give one example where constraints are a better fit than pure backtracking.
- Compare Prolog querying to SQL: what is similar and what is different?